

Independent Component Analysis

Joshua Frankie Rayo

2011-19612

CS 280 Programming Assignment 4

November 10, 2016

Contents

1	Introduction	1
2	Methodology	1
2.1	Input Data	1
2.2	Running the FastICA algorithm	2
3	Results	3

1 Introduction

In this paper the independent component analysis was used to separate the source signals from a set of observations, assuming linear mixture and the sources are statistically independent. Five sound observations are the input data and ICA algorithm outputs the five independent sound sources. Several factors were taken into account relevant to the algorithm: input sampling rate, centering, whitening and appropriate contrastive function. Finally, the residual was used to measure the reconstruction performance of the algorithm.

2 Methodology

The code is made using Python, a high level, general-purpose programming language that is relatively easy to learn and has very good features in list manipulation. The `sklearn` library of Python was used to execute the FastICA algorithm. Other libraries used are `numpy` and `scipy`.

2.1 Input Data

The audio files `mic[1-5].wav` contains synchronized audio recordings captured by five microphones positioned at five different locations. Each sound file contains

mixture of 5 independent signals with varying volumes. Each sound file was read using `scipy.io.wavfile.read()` that determines its sampling rate and wave sequence of length 191258. The sampling rate is 22050 Hz. The mixture matrix was formed by augmenting the wave sequences, which as of size 191258x5.

2.2 Running the FastICA algorithm

Different parameters are necessary to run the algorithm, listed below:

- appropriate input sampling rate
- whether centering is required for preprocessing
- whether whitening is necessary for preprocessing
- appropriate contrastive function $G(y)$

these parameters are experimented to determine which produces the best un-mixing of the sources.

There are several techniques to convert sampling rate from one to another, including over-sampling, downsampling and decimation. Several papers argue that oversampling results in redundant and irrelevant signal information. Redundant information may affect the ICA by "conceding" some source components under the identity of artifacts and requires more processing time and memory. A simple filtering still does not remove the redundant information. Two approaches were suggested by [1]: *Resample-ICA* and *Modified Wavelet-ICA* but this is already out of the scope. For the input sampling rate, it was kept at its original value.

Different instances of ICAs were created with varying contrastive function, enabling/disabling centering and whitening. The list below summarizes the possible parameter values on an ICA:

- Centering: **True** or **False**
- Whitening: **True** or **False**
- Contrastive functions:

$$\begin{aligned}
 - g_1(y) &= \frac{1}{a} \log \cosh(a * y) \\
 - g_2(y) &= \frac{1}{a} \exp\left(-a * \frac{y^2}{2}\right) \\
 - g_3(y) &= \frac{1}{4}y^4
 \end{aligned}$$

A total of 12 ICAs are initialized with varying parameters were run for 50 times to measure their mean residual. The residual is computed as the difference between the original and reconstructed observations from the ICA. The assumed mixing model is $X = AS$, the outputs of ICA will be $\dot{X} = \dot{A}\dot{S}$. The residual matrix will now be $R = \text{abs}(X - \dot{X})$ and the residual is now $r = \|\dot{R}\|_2$, the 2-norm of R . The ICA with lowest mean residual will be used to produce the independent signals and reconstruct the observations.

3 Results

After running the ICAs repeatedly, the lowest mean residual was produced with centering and whitening both enabled with g_1 as the contrastive function. Similar runs are given below.

```

scl storge@ubuntu:~/Downloads/Audio_Data$ python -u pa.py
0
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
1
0/usr/local/lib/python2.7/dist-packages/sklearn/decomposition/fastica_.py:290: UserWarning: Ignoring n_components with whiten=False.
  warnings.warn('Ignoring n_components with whiten=False.')
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
2
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
3
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
4
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
5
0/usr/local/lib/python2.7/dist-packages/sklearn/decomposition/fastica_.py:116: UserWarning: FastICA did not converge. Consider increasing to
the maximum number of iterations.
  warnings.warn('FastICA did not converge. Consider increasing '
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
6
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
7
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
8
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
9
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
10
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
11
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
ICA: fun - logcosh, centering - True, whiten - True : 2774444.28513

```

Figure 1: the best ICA is determined after 50 runs

```

franco@franco:~/Downloads/Audio_Data$ python -u pa.py
0
0 1 2 3 4 5 6 7 8 9
1
0/usr/local/lib/python2.7/dist-packages/sklearn/decomposition/fastica_.py:290: UserWarning: Ignoring n_components with whiten=False.
  warnings.warn('Ignoring n_components with whiten=False.')
1 2 3 4 5 6 7 8 9
2
0 1 2 3 4 5 6 7 8 9
3
0 1 2 3 4 5 6 7 8 9
4
0 1 2 3 4 5 6 7 8 9
5
0/usr/local/lib/python2.7/dist-packages/sklearn/decomposition/fastica_.py:116: UserWarning: FastICA did not converge. Consider increa
the maximum number of iterations.
  warnings.warn('FastICA did not converge. Consider increasing '
1 2 3 4 5 6 7 8 9
6
0 1 2 3 4 5 6 7 8 9
7
0 1 2 3 4 5 6 7 8 9
8
0 1 2 3 4 5 6 7 8 9
9
0 1 2 3 4 5 6 7 8 9
10
0 1 2 3 4 5 6 7 8 9
11
0 1 2 3 4 5 6 7 8 9
ICA: fun - logcosh, centering - True, whiten - True : 3869740.87326

```

Figure 2: the best ICA is determined after 10 runs

The FastICA algorithm might not converge to a solution given function g_3 , centering disabled and whitening enabled. To get results with low residual, centering is required, whitening is necessary for preprocessing and the appropriate contrastive function is $g_1(y) = \frac{1}{a} \log \cosh(a * y)$. The residual values are also printed when the program is run. In general, whitening and centering is not necessary for FastICA, but may produce larger residual.

The independent files are now obtained including one with a male person speaking and the other is an orchestral music. The independent sounds are significantly lesser in volume than the signals from the original observations.

References

- [1] P. Krishna, M. Rajasekhara, E. Ariwa, *Global Trends in Information Systems and Software Applications*, 4th ed., 2011